Deep Learning

03-06-26

- In ANN, during backprop we compute gradients and update weights using them

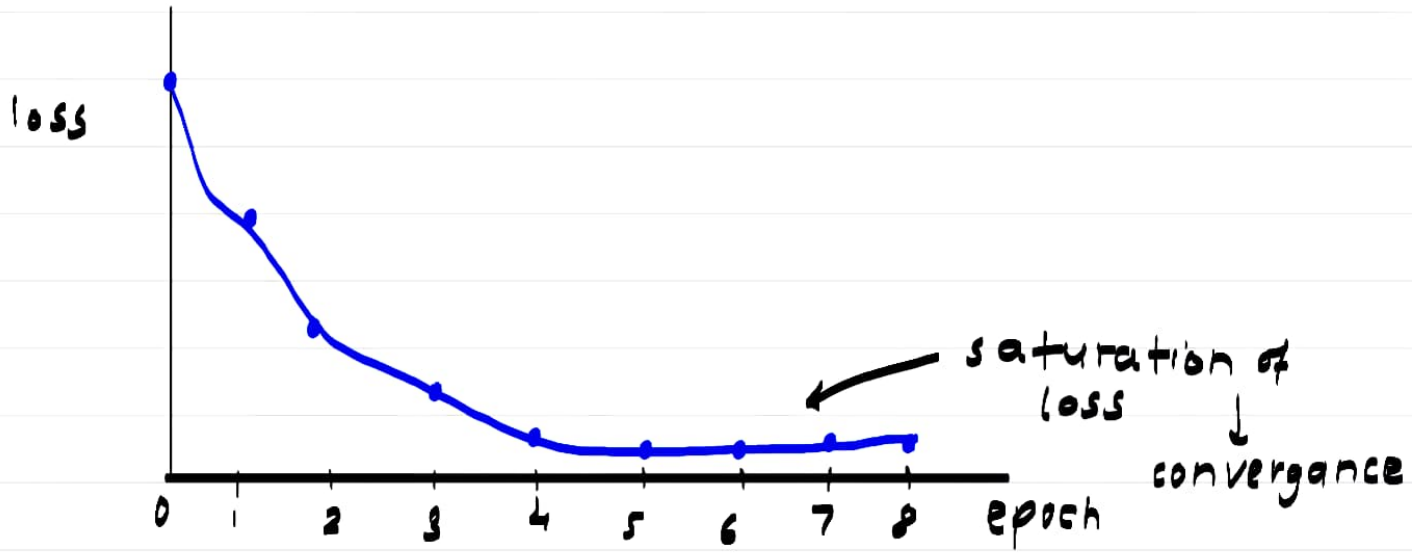
- When ANN is getting trained - one forward pass + one backward pass is called one training epoch.

one epoch = one forward pass + one backward pass

Number of epochs is a training parameter we need to decide before ANN training starts.

- During training, we track the loss wrt no. of epochs

- Training stops after convergence.



- All operations in ANN happen in matrix form
  forward pass, loss computation, gradient computation
  weight update

Why doing these operations in matrix format is important?

In ANN regression

mse loss: $L = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$

```
sum of squares = 0
for i in range(n):
    sumofsquares += (yi - ŷi)^2
mse = sum of squares / n
```

loss computation without matrix form which happens in series inside CPU

↓

In series means processing only 1 datapoint at a time - for loop

Processing in series makes the operations very very slow

matrix form allows these operations to happen parallely inside GPU - which is very very fast

$L = \frac{1}{n} [(y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2 + (y_3 - \hat{y}_3)^2 \ldots (y_n - \hat{y}_n)^2]$

lets see how to do it in matrix format

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} \qquad \hat{Y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \\ \vdots \\ \hat{y}_n \end{bmatrix} \qquad Y - \hat{Y} = \begin{bmatrix} y_1 - \hat{y}_1 \\ y_2 - \hat{y}_2 \\ y_3 - \hat{y}_3 \\ \vdots \\ y_n - \hat{y}_n \end{bmatrix}$$

$$(Y - \hat{Y})^T = \begin{bmatrix} (y_1 - \hat{y}_1) & (y_2 - \hat{y}_2) & (y_3 - \hat{y}_3) & \dots & (y_n - \hat{y}_n) \end{bmatrix}$$

$$L = \frac{1}{n} (Y - \hat{Y})^T (Y - \hat{Y})$$

$(Y - \hat{Y})^T$: $1 \times n$, $(Y - \hat{Y})$: $n \times 1$

$$L = \frac{1}{n} \times \left[(y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2 + (y_3 - \hat{y}_3)^2 \dots (y_n - \hat{y}_n)^2 \right]$$

$L = \frac{1}{n} \times$ sum of squares

GPU - Graphical Processing Unit

GPU - 1000+ cores CPU - 16 / 32 cores

As GPU has lot many cores compared to CPU, its specialised in parallel processing

Nvidia - 5 Trillion → designs GPUs

When model is very huge - model having a lot of weights the weight matrix W would not fit in GPUs ram.

- 7 Billion weights can't fit on 16 GB GPU

to solve this we need to go for higher ram GPU

When datasize 'n' is very large

the complete data matrix X would not fit into GPU's ram

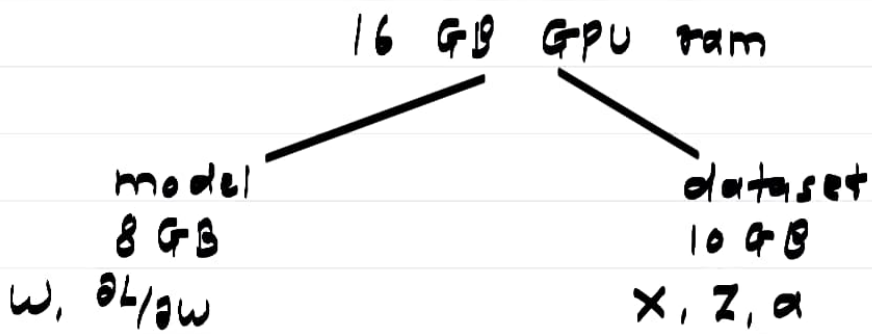
solution → batch processing

'n' → $\frac{n}{10}$, $\frac{n}{10}$, $\frac{n}{10}$ .... to batches

do forward + backward pass on single batch at a time

$$Z = WX \left\{ \begin{array}{l} Z_1 = WX_1 \\ Z_2 = WX_2 \\ Z_3 = WX_3 \\ \vdots \\ Z_{10} = WX_{10} \end{array} \right.$$

Gradient descent will also happen for one batch at a time → this is called minibatch gradient descent



if we divide dataset into 5 batches

each batch will need $\frac{10}{5} = 2$ GB

Mini Batch GD :

we need to decide batch size first

eg. 100 datapoints

batch size = 20 every batch will have 20 datapoints

no. of batches = $\frac{100}{20} = 5$

to train the model

- initialize model weights with random values
- in 1st step do forward and backward propagation for 1st batch - compute loss, gradients, outputs, etc.

$Z = wB_1$ B_1 - data matrix for a batch 1

$a = f(z)$

$\hat{y} = a$

$\longrightarrow$ loss $\longrightarrow \frac{\partial L}{\partial w} \longrightarrow$ update w

- Repeat these steps for all batches

eg. randomly initialized weights $w^{(0)}$

$B_1 \longrightarrow$ forward + backward

$Z = w^{(0)} B_1$ $a = f(z)$ $L \longrightarrow \partial L / \partial w^{(0)}$

$w^{(1)} = w^{(0)} - \alpha \, \partial L / \partial w^{(0)}$ $\longrightarrow$ 1st weight updation

$B_2 \rightarrow z = w^{(1)} B_2 \quad a = f(z) \quad L \rightarrow \partial L / \partial w^{(1)}$

$w^{(2)} = w^{(1)} - \alpha \, \partial L / \partial w^{(1)}$ → 2nd weight updation

for 5 batches - weights will be updated 5 times

↳ 5 train steps

in this case

1 train epoch = 5 train steps

100 datapoints

batch size = 20

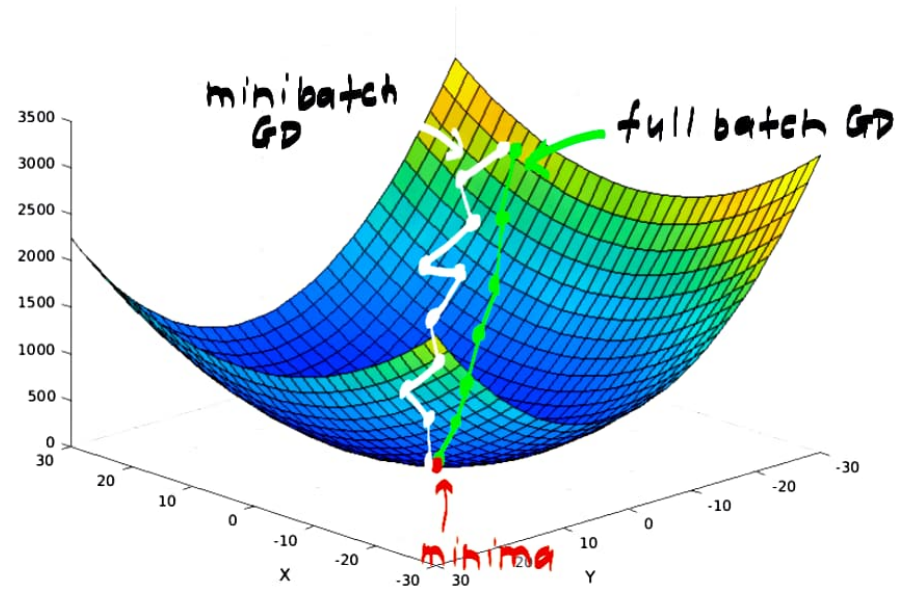
no. of batches = 5

no. of epochs = 3

how many times weight updation will happen?

→ 5 weight updations in every epoch

total weight updations = 5x3 = 15



full batch GD - smooth training

mini batch GD - noisy training

} same minima after convergance

Stochastic Gradient Descent: SGD